

STRINGS

1. Write a program to count the number of vowels in a given string.
2. Given a string, print the reverse of the string without using slicing.
3. Check whether a given string is a palindrome (ignore case and spaces).
4. Write a program to find the most frequent character in a string.
5. Given a paragraph, remove all duplicate words while preserving the original order.

USER INPUT

1. Take two numbers as input and print their sum.
2. Take a string as input and print its length.
3. Take comma-separated numbers as input and convert them into a list of integers.
4. Ask the user for name, age, and city and print a formatted string:
`"Hello <name>, you are <age> years old from <city>!"`
5. Take a sentence as input and print each word on a new line sorted by length.

DATA STRUCTURES (List, Set, Tuple, Dictionary)

1. Create a list of numbers and print the maximum and minimum values.
2. Create a dictionary with 3 key-value pairs and print all keys.
3. Given a list with duplicates, convert it to a set and remove duplicates without losing original order.

4. Write a program to merge two dictionaries; if a key repeats, add their values.
5. Given a list of tuples (`name`, `score`), return a dictionary grouping all names with the same score.

FLOW CONTROL (if-else-elif)

1. Check whether a number is positive, negative, or zero.
2. Check whether a given character is a vowel or consonant.
3. Take marks as input and print grade (A/B/C/D/Fail).
4. Check whether a year is a leap year using if-elif conditions.
5. Write a program that takes three numbers and prints the largest without using `max()`.
6. Flight Ticket Classification Using Multiple Conditions

A flight ticket is assigned a category based on:

- If ticket price > 15,000 and duration < 3 hr → “Business Priority”
- Else if price > 10,000 → “Business Saver”
- Else if price > 5,000 → “Economy Flex”
- Else → “Economy Basic”

Modifiers:

- If the passenger is a loyalty member with >3 years tenure → upgrade category by one level
- If the travel date is a holiday → downgrade category by one level
- Categories cannot go above “Business Priority” or below “Economy Basic”

Create a program using if–elif–else to determine the final ticket category

7. Advanced Tax Computation With Conditional Rebates

Income tax rules:

- Income $\leq 2.5L \rightarrow$ No tax
- Income $\leq 5L \rightarrow 5\%$
- Income $\leq 10L \rightarrow 20\%$
- Above 10L $\rightarrow 30\%$

Additional rules:

- If the individual is a senior citizen (age ≥ 60), reduce tax rate by 2%
- If they have health insurance premium receipt $\rightarrow$ subtract ₹15,000
- If final tax $< 0 \rightarrow$ make it 0
- If tax $\geq 1,00,000 \rightarrow$ apply 4% cess

Using nested and sequential if-elif-else statements, calculate the final tax

8. Admission Decision Based on Multiple Criteria

A college gives admission based on:

- If entrance score $\geq 90 \rightarrow$ “Selected – Merit”
- Else if score ≥ 75 and 12th marks $\geq 80 \rightarrow$ “Selected – General List”
- Else if score ≥ 60 and extracurricular score $\geq 8 \rightarrow$ “Selected – Sports/Arts Quota”

- Otherwise → “Rejected”

Special cases:

- If candidate is from an economically weaker section and meets at least one cutoff within 5 marks → upgrade decision to the next higher category.
- If candidate has disciplinary issues → automatically downgrade to “Rejected”.

Write the full decision logic using if–elif–else.

9. Electricity Billing With Tiered Slabs + Penalties

An electricity board uses the following slab rates:

- First 100 units → ₹3 per unit
- Next 200 units → ₹5 per unit
- Above 300 units → ₹8 per unit

Additionally:

- If usage > 500 units → add a 15% surcharge
- If bill amount > ₹2000 → add environmental tax of ₹150
- If user is a rural consumer, give a 10% discount on final bill

Write a program using multiple if–elif–else conditions to calculate the final bill.

10. Multi-Tier Pricing With Conditional Adjustments

A store provides the following pricing rules:

- If total purchase $\geq ₹5000 \rightarrow 20\%$ discount
- Else if total purchase $\geq ₹3000 \rightarrow 12\%$ discount
- Else if total purchase $\geq ₹1000 \rightarrow 5\%$ discount
Additionally:
- If customer is a Premium Member, apply an extra 5% discount
- If the purchase day is Sunday, apply a flat ₹100 off after all discounts
- If the final price goes below ₹500 $\rightarrow$ set it to ₹500 (minimum payable)

Write a program using an if-elif-else chain to compute the final payable amount.

FOR LOOP (Break, Continue, Nested Loop)

1. Print numbers from 1 to 20 but skip multiples of 3 (use continue).
2. Print elements of a list one by one using a for loop.
3. Find the first number divisible by 7 in a list (use break).
4. Create a multiplication table (nested loop) from 1 to 5.
5. Given a list of lists, flatten it using nested for loops.
6. Dynamic Filtering with Nested Loops & Continue

Given a list of lists containing numbers, print only the numbers that satisfy:

- Even number
- Not divisible by 5
- And greater than the average of all numbers in that sub-list
Use nested loops + `continue` to skip invalid values.

7. Pattern Printing With Conditional Break

Print the following pattern for N (input as string), but stop generating further rows if the sum of numbers in any row exceeds 30:

Example for N=6:

```
6. 1
7. 2 3
8. 4 5 6
9. 7 8 9 10
10. ...
```

Use nested loops + break.

8. Find Pair With Target Sum (Double Nested Loop + Early Termination)

Given a list of numbers and a target value T, find the first pair (a, b) such that $a + b = T$.

Use nested loops.

Terminate inner loop using `break` once the pair is found, and outer loop using a flag.

9. Matrix Diagonal Extraction Using Nested Loops

Given a square matrix (list of lists), extract:

- Main diagonal
- Secondary diagonal
Ignore all elements that are multiples of 4 using `continue`.
Use nested for loops.

10. Generate Prime Numbers in a Matrix Form (Nested Loop + Continue)

Input is a string N (for matrix size).

Fill an $N \times N$ matrix row-wise with numbers from 1 to N^2 .

While printing, skip non-prime numbers using `continue`.

Print only the primes in matrix form (empty spots allowed as _).

WHILE LOOP (Break, Continue, Nested Loop)

1. Print numbers from 1 to 10 using a while loop.
2. Print even numbers from 1 to 20 using while + continue.
3. Take input until user enters "stop".
4. Reverse a number using while loop.
5. Implement a number guessing game with limited attempts (use break).
6. Rotate List Elements Until Sorted (While + Break)

Given a list of integers, repeatedly rotate it left by one position until the list becomes strictly increasing.

Stop when sorted OR after 20 rotations (whichever comes first). Use while + break.

7. Generate Collatz Sequence With While + Conditions

Given a positive integer N, generate its Collatz sequence using while:

- If N is even: $N = N/2$
- If N is odd: $N = 3N + 1$
Stop when N becomes 1.
Skip steps where N becomes divisible by 5 using continue and still proceed to the next step.

8. Multi-Level Loop: Password Validator With Nested While

User gets 3 main attempts.

Inside each attempt, you give them 5 sub-attempts to enter a valid password.

Use nested while loops + break + continue.

Password rules:

- At least 10 characters
- Contains uppercase, lowercase, digit, and special symbol.

Stop immediately when a valid password is created.

9. Simulate Elevator Movement (While + Continue)

An elevator starts at floor 0.

Input is a string of commands like: "UP 5, DOWN 2, UP 3, DOWN 10"

Rules:

- Elevator cannot go below 0 → ignore (continue) such command
- Stop processing if elevator reaches floor 20 (break)
Return final floor.

10. Nested While Loop: Pattern Searching in a String

Given a long string S and a pattern P, use nested while loops to scan S and find all starting indices where P occurs. Skip indices where the remaining string is shorter than P using continue. Cannot use Python's built-in search functions.

Example: String S: "ababcabcbabc"

Pattern P: "abc"

FUNCTIONS :

1. Write a function without arguments that prints today's date.
2. Write a function that takes two numbers as arguments and returns their product.
3. Write a function that takes a list and returns only even numbers.

4. Write a function that takes a string argument and returns a dictionary of character frequencies.
5. Write a function that accepts variable number of arguments (*args) and returns the sum of only integers.
6. Write a function that takes a list of integers and returns three values:
 - A list of prime numbers
 - A list of composite numbers
 - A list of numbers that are neither (0, 1, negatives)The function must not use any built-in prime-checking libraries.
7. Write a function `bulk_price(unit_price, quantity)` that applies:
 - 5% discount if quantity ≥ 10
 - 10% discount if quantity ≥ 25
 - 20% discount if quantity ≥ 50
 - Consider any amount of your choice as unit price.

Then add GST 12% to the discounted total.
Return the final payable amount.

8. Write a function `calculate_final_price(product_price, discount_list, customer_type)` that calculates the final payable amount after applying:
 - A list of percentage discounts (apply in order)
 - An additional flat ₹200 discount if customer_type = "premium"
 - GST of 18% added after all discounts
 - If final price goes below ₹100, set it to ₹100 (minimum bill rule).
Return the final price.

9. Write a function that takes a list of strings and returns a new list containing:
- Strings converted to uppercase
 - Only those whose length is a prime number
- You must use `map`, `filter`, and a lambda expression inside the function.
10. Write a function that takes a list which may contain:
- integers
 - lists
 - nested lists of any depth
- Return the total sum of all integers found at any depth without using built-in flatten methods.

LIST COMPREHENSIONS:

1. Create a list of squares from 1 to 10 using list comprehension.
2. Return a list of even numbers from a given list.
3. Replace vowels in a string with '*' using list comprehension.
4. Given a list of words, return words having length > 4 using list comprehension.
5. Flatten a 2D matrix using list comprehension only.

LAMBDA FUNCTIONS

1. Write a lambda to add two numbers.
2. Write a lambda to find the square of a number.
3. Sort a list of tuples based on the second value using lambda.
4. Use lambda to extract last names from a list of full names.

5. Use `lambda + sorted()` to sort a list of dictionaries by nested key:
Example: `{"name": "A", "score": {"math": 90}}`

FILTER:

1. Filter out even numbers from a list.
2. Filter words that start with the letter 'a'.
3. Filter out numbers divisible by both 3 and 5.
4. From a list of strings, filter those containing digits.
5. Given a list of dictionaries of students, filter those who scored above class average.

MAP:

1. Convert a list of strings to uppercase using `map`.
2. Convert a list of numbers to their cubes using `map`.
3. Add 10 to every number in the list using `map + lambda`.
4. Convert a list of integers to strings using `map`.
5. Given two lists, use `map` to add corresponding elements (like vector addition).

REDUCE:

1. Use `reduce` to find the sum of elements in a list.
2. Use `reduce` to find the maximum number in a list.

3. Use reduce to concatenate a list of strings into a single string.
4. Use reduce to multiply all elements in a list.
5. Use reduce to find the longest word in a list of sentences after splitting them.

Advance Questions:

1. Write a python program to verify if a given string s represents a valid number according to certain rules defined for decimal numbers and integers.
 - i. A valid number can be an integer or a decimal number.
 - ii. It can optionally be followed by 'e' or 'E', followed by an integer.
 - iii. A decimal number can optionally start with a sign character, '+' or '-'.
 - iv. The decimal part can have one or more digits followed by a dot or a dot followed by one or more digits.
 - v. An integer can optionally start with a sign character, '+' or '-'.

Valid examples include: "2", "0089", "-0.1", "4.", "2e10", "3e+7", "53.5e93", etc.

Invalid examples are: "abc", "1a", "1e", "--6", "95a54e53", etc.

Input:

'-0.1'

Output :

"Valid Number"

Input:

'abc'

Output :

"not Valid Number"

2. Wildcard Matching Questions

Given two strings 'S1' and 'S2'. String S1 can have the following two special characters:

- i. '?' can be matched to any single character of S2.
- ii. '*' can be matched to any sequence of characters of S2. (sequence can be of length zero or more).

We need to check whether strings S1 and S2 match or not.

Input :

S1 : '*iger'

S2 : 'Tiger'

Output: Matches (Here ? can be matched with R Single character)

Input :

S1 : '**def'

S2 : 'def'

Output: Matches (Here * can be matched with any sequence of length 0 or more)

Input :

S1 : 'fgik'

S2 : 'fg?d'

Output : Doesn't Matches (character k in 4th index position of s1 does not match with character d in 4th index position of s2).

3. Given an array arr containing N integers, find the sum of the absolute difference of maximum and minimum of all subarrays.

Input : arr[] = (1,5,2)

Output : 11

Explanation : The following are the six subarrays:

1. [1] : max - min = 1-1 = 0
2. [5] : max - min = 5-5 = 0
3. [2] : max - min = 2-2 = 0
4. [1,5] : max - min = 5-1 = 4
5. [5,2] : max - min = 5-2 = 3
6. [1,5,2] : max - min = 5-1 = 4

Final result will be $0+0+0+4+3+4 = 11$

4. Create user defined function that validates a password and encrypts it then a separate function that decrypts the same(10 Marks)

- password must have:
 1. minimum 8 characters
 2. atleast 1 uppercase alphabet
 3. atleast 1 lowercase alphabet
 4. atleast 1 special character
 5. atleast 1 number

-
- Encryption:
 1. The encryption function accepts the password of length n and every character is changed with a different character of ascii value of character + n.

-
- Decryption
 2. The decryption function accepts the encrypted password of length n and should display the original password entered by user.

5. Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Every close bracket has a corresponding open bracket of the same type.

Input :

'()'

Output:

Valid

Input:

'()[]{}'

Output:

Valid

Input:

'[]'

Output:

Invalid

6.Maximum Sum of Two Non-Overlapping Subarrays

Given an integer array and two integers firstLen and secondLen, return the maximum sum of elements in two non-overlapping subarrays with lengths firstLen and secondLen.

The array with length firstLen could occur before or after the array with length secondLen, but they have to be non-overlapping.

A subarray is a contiguous part of an array.

Input : arr = [3,7,2,1,5,4,9]

$k = 2$

Output :

3 7

4 9

(3,7) & (4,9) are the 2 possible subarrays whose sum is maximum among all the possible combinations.

7. The set $[1, 2, 3, \dots, n]$ contains a total of $n!$ unique permutations.

By listing and labeling all of the permutations in order, we get the following sequence for $n = 3$: "123"

"132"

"213"

"231"

"312"

"321"

Given n and k , return the k th permutation sequence.

Example 1:

Input: $n = 3, k = 3$

Output: "213"

Example 2:

Input: $n = 4, k = 9$

Output: "2314"

Example 3:

Input: $n = 3, k = 1$

Output: "123"

Constraints:

$1 \leq n \leq 9$

$1 \leq k \leq n!$

8. A newly opened multinational brand has decided to base its company logo on the three most common characters in the company name. They are now trying out various combinations of company names and logos based on this condition. Given a string S , which is the company name in lowercase letters, your task is to find the top three most common characters in the string.

1. Print the three most common characters along with their occurrence count.
2. Sort in descending order of occurrence count.
3. If the occurrence count is the same, sort the characters alphabetically.

For example, according to the conditions described above, GOOGLE would have its logo with the letters G, O, E.

Sample Input:

AABBBCCDE

Sample Output:

B A C

Explanation:

AABBBCCDE

Here, B occurs 3 times. It is printed first.

Both A and C occur 2 times. So, A is printed in the second line and C in the third line because A comes before C in the alphabetical order.

Note: The string S should have atleast 3 distinct characters.

9. Python program to get all possible slices of a string for K number of slices. Given a string, the task is to write a Python program to get all possible slices for the K number of slices.

Input:

String = "DSECOURSE",

K = 3

Output: [['D', 'S', 'ECOURSE'], ['D', 'SE', 'COURSE'], ['D', 'SEC', 'OURSE'], ['D', 'SECO', 'URSE'], ['D', 'SECOU', 'RSE'], ['D', 'SECOUR', 'SE'], ['D', 'SECOURS', 'E'], ['DS', 'E', 'COURSE'], ['DS', 'EC', 'OURSE'], ['DS', 'ECO', 'URSE'], ['DS', 'ECOU', 'RSE'], ['DS', 'ECOUR', 'SE'], ['DS', 'ECOURS', 'E'], ['DSE', 'C', 'OURSE'], ['DSE', 'CO', 'URSE'], ['DSE', 'COU', 'RSE'], ['DSE', 'COUR', 'SE'], ['DSE', 'COURS', 'E'], ['DSEC', 'O', 'URSE'], ['DSEC', 'OU', 'RSE'], ['DSEC', 'OUR', 'SE'], ['DSEC', 'OURS', 'E'], ['DSECO', 'U', 'RSE'], ['DSECO', 'UR', 'SE'], ['DSECO', 'URS', 'E'], ['DSECOU', 'R', 'SE'], ['DSECOU', 'RS', 'E'], ['DSECOUR', 'S', 'E']]

Explanation: All possible 3 slices for constructing string are returned.

10. Given a list of integers and an integer target, return indices of the two numbers such that they add up to the target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.

Input: nums = [2,7,11,15], target = 9

Output: [0,1]

Explanation: Because $\text{nums}[0] + \text{nums}[1] == 9$, we return [0, 1].

11. Write a function to find the number of weekdays in a given month & year. The function can accept two parameters, and both will be numbers. The first parameter will be the number indicating the month, and the second will be the year in four digits. Your function should parse the parameters, and it must return the required output.

Input:

Month = 6

Year = 2024

Output : x`23

12. Define a function that can generate a dictionary where the keys are numbers between 1 and 20 (both included) and the values are squares of keys.

13. Python program to Sort a List of Dictionaries by the Sum of their Values
Given the Dictionary List, sort by summation of their values in ascending order.

Input : list1 = [{1 : 3, 4 : 5, 3 : 5}, {1 : 100}, {8 : 9, 7 : 3}]

Output : [{8 : 9, 7 : 3}, {1 : 3, 4 : 5, 3 : 5}, {1 : 100}]

Explanation: $12 < 13 < 100$, sorted by values sum

14. Python program to sort Palindrome Words (alphabetical order) in a Sentence
Given a string S representing a sentence, the task is to reorder all the palindromic words present in the sentence in sorted order.

Input: S = "Please refer to the madam to know the level"

Output: Please level to the madam to know the refer

Explanation: Here "refer", "madam", and "level" are the palindromic words. Sorting them generates the sequence {"level", "madam", "refer"}.

15. Longest Palindromic Substring

Given a string s, return the longest palindromic substring in s.

Input: s = "cbabd"

Output: "bb"

16. Find the Longest Substring Without Repeating Characters

From the given string S, find the length of the longest substring without repeating characters.

Example 1:

Input: s = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with a length of 3.

17. You are tasked with developing a program for an educational institute to manage student admissions. The program should collect the following information for each student:

A unique enrollment ID

Name

Address

Phone number

Highest qualification

The program should continue to accept student information until no input is provided (i.e., the user presses Enter without entering any data).

You should use a dictionary to store this data, with the enrollment ID as the key. Once all entries are complete, the program should display the list of enrolled students along with the total count of students

18.This task involves an English adaptation of the Japanese word game Shiritori. The objective is to adhere to two specific rules for valid word sequences:

The first character of the next word must match the last character of the previous word.

A word must not have been previously used in the sequence.

For example:

Valid Sequence: ["word", "dowry", "yodel", "leader", "righteous", "serpent"]

Invalid Sequence: ["motive", "beach"] (Reason: "beach" should begin with "e")

Invalid Sequence: ["hive", "eh", "hive"] (Reason: "hive" has already been used)

Write a program to evaluate the given sequences based on these rules.

The program should only break when there is a invalid sequence.

19. You are given a task to write a function to find all the pairs in a list whose sum is equal to a given value.

Sample Input:

[1,2,3,4,5,6,7,8,9]

Sample Output:

[(1,9),(2,8),(3,7),(4,6)]

20.Write a Python function that finds all the permutations of the members of a list.

Sample Input:

[1,2,3]

Possible Permutations of the passed list:

[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]